# Facial Emotion Recognition

Deep Learning Final Project - Team Report

Repo: github.com/Rehanabbaxi/Facial-Emotion-Recognition | Framework: PyTorch | Runtime: Google Colab (T4 GPU) | Dataset: FER2013

## 1. What this project does

We build a system that looks at a face image and predicts the emotion it shows, choosing one of seven categories: angry, disgust, fear, happy, neutral, sad, surprise. The full pipeline - data, training, evaluation and live inference - lives in one Colab notebook (FER_Colab_Notebook.ipynb) that runs end to end on a free GPU with no path edits.

Rather than train a single network, we benchmark THREE models to compare design choices (a custom network vs a pretrained one, different optimizers, and different ways of handling class imbalance) and then pick the best. All figures and numbers below are the actual outputs from our Colab run.

## 2. The dataset (FER2013)

- 35,887 grayscale face images, each 48x48 pixels, labelled with one of 7 emotions (28,709 train / 7,178 test).
- Strongly IMBALANCED: "happy" has 7,215 images while "disgust" has only 436 - a 16.5x gap. A naive model just learns the common classes and ignores the rare ones.
- Known weaknesses: low resolution, grayscale, and some mislabelled / ambiguous faces. This is why even strong models top out around 70-75% on FER2013.

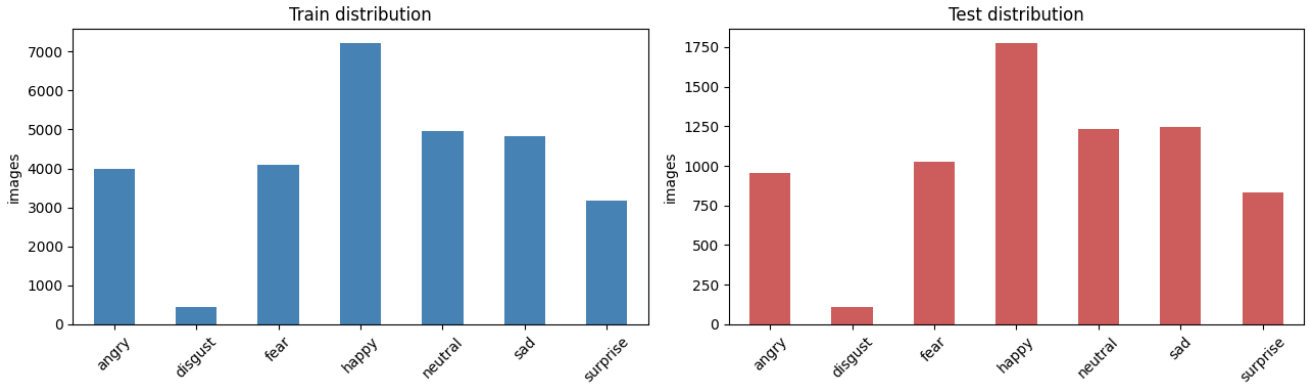



*Figure 1. Class distribution of the train (left) and test (right) sets. The severe imbalance (happy >> disgust) is what motivates class weighting, weighted sampling and focal loss later.*

## 3. Preprocessing and data augmentation

Images are normalised for stable training. During training we randomly alter images so the model generalises - but only with changes that do NOT destroy the expression:

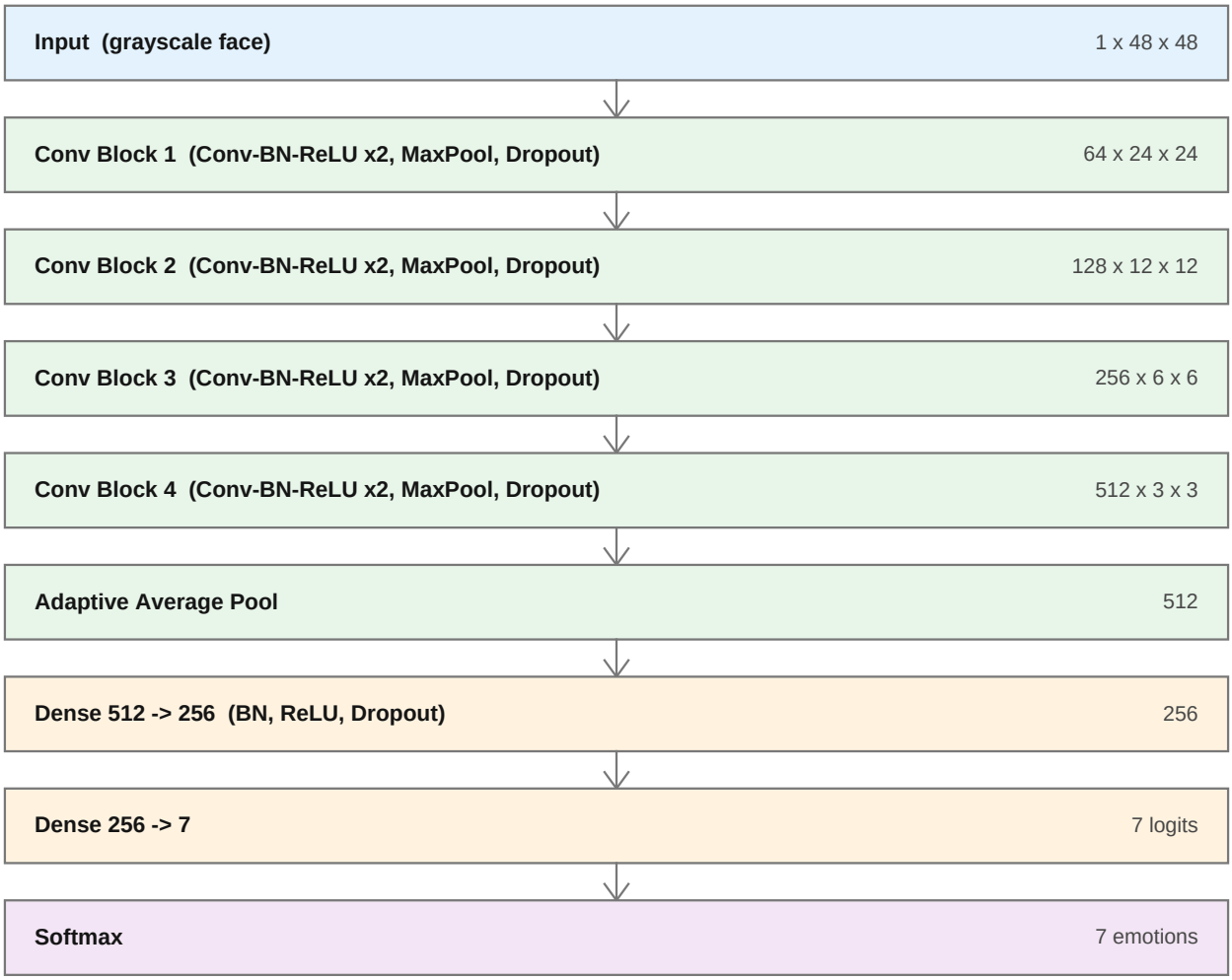
- Used: horizontal flip, small rotation (+/-12 deg), slight zoom/shift, brightness/contrast jitter, light random erasing (simulated occlusion).
- Avoided: vertical flip and large rotations - an upside-down face is not a real expression.

# 4. The three models

All three trained models are saved in the wieghts/ folder. Below is what each one is, its architecture diagram, and its training curves.

### Model 1 - Custom CNN baseline (ckpt_cnn_baseline.pt)

A convolutional network we built from scratch (VGG-style: four blocks of Conv-BatchNorm-ReLU with pooling and dropout, then a small dense classifier). It learns features directly from the 48x48 grayscale images and is our reference point.




Trained with AdamW + cosine learning-rate decay + class-weighted cross-entropy. Result: test accuracy 63.5%, macro-F1 0.588.

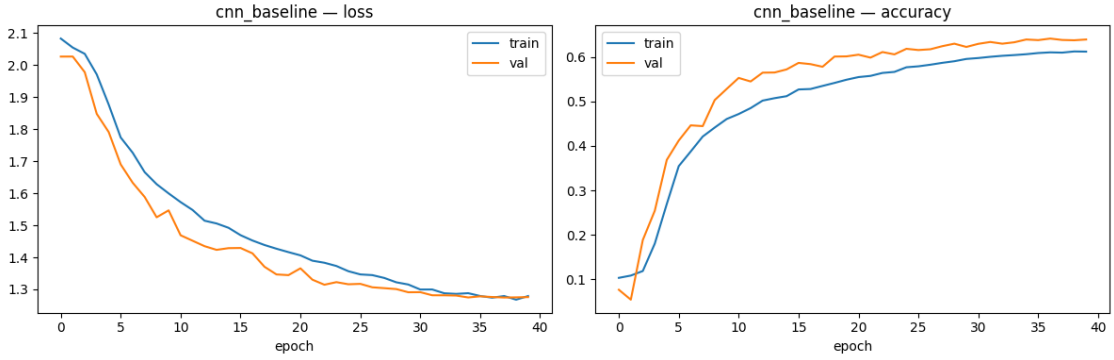
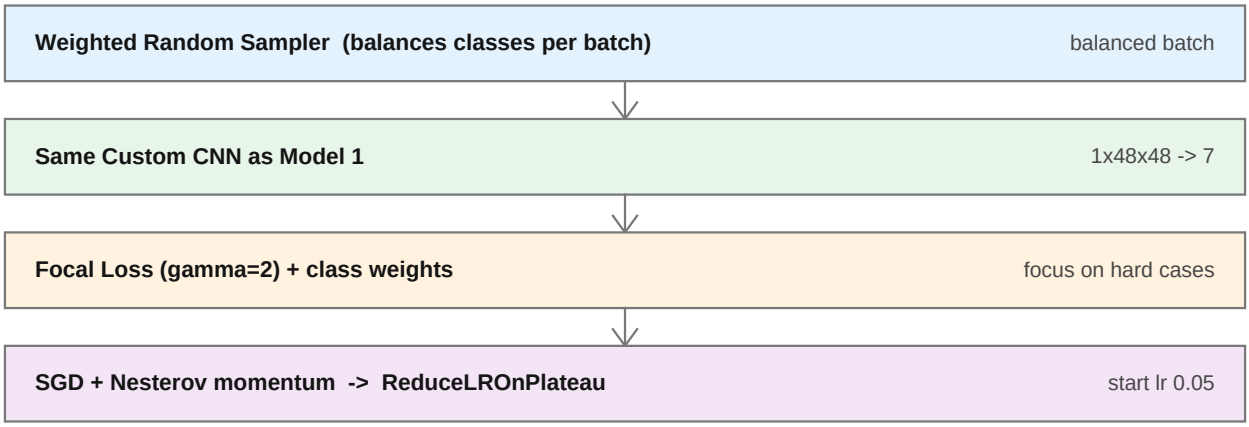



*Figure 2. Model 1 training curves - smooth, healthy convergence over 40 epochs.*

## Model 2 - Custom CNN + Focal Loss + SGD (ckpt_cnn_focal_sgd.pt)

SAME network as Model 1, but trained with a different strategy aimed squarely at the imbalance: a weighted sampler balances each batch, focal loss makes the model focus on hard/rare examples, and it is optimised with SGD+momentum instead of AdamW. The diagram below shows the training pipeline (the network itself is identical to Model 1).




Honest finding: this config UNDER-performed (test accuracy 53.2%, macro-F1 0.474 - the worst of the three). The curves show why: SGD at lr 0.05 stalled near 1.5% validation accuracy for ~8 epochs before it started learning, and never caught up in the epoch budget. Lesson for the team: focal loss is not a guaranteed win - it is sensitive to the optimizer and learning rate, and here a well-tuned weighted cross-entropy (Model 1) beat it. This is exactly why we benchmark instead of assuming.

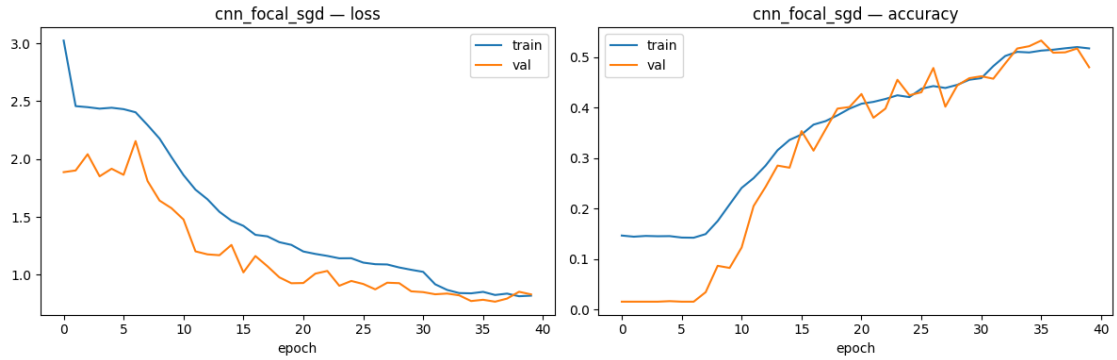
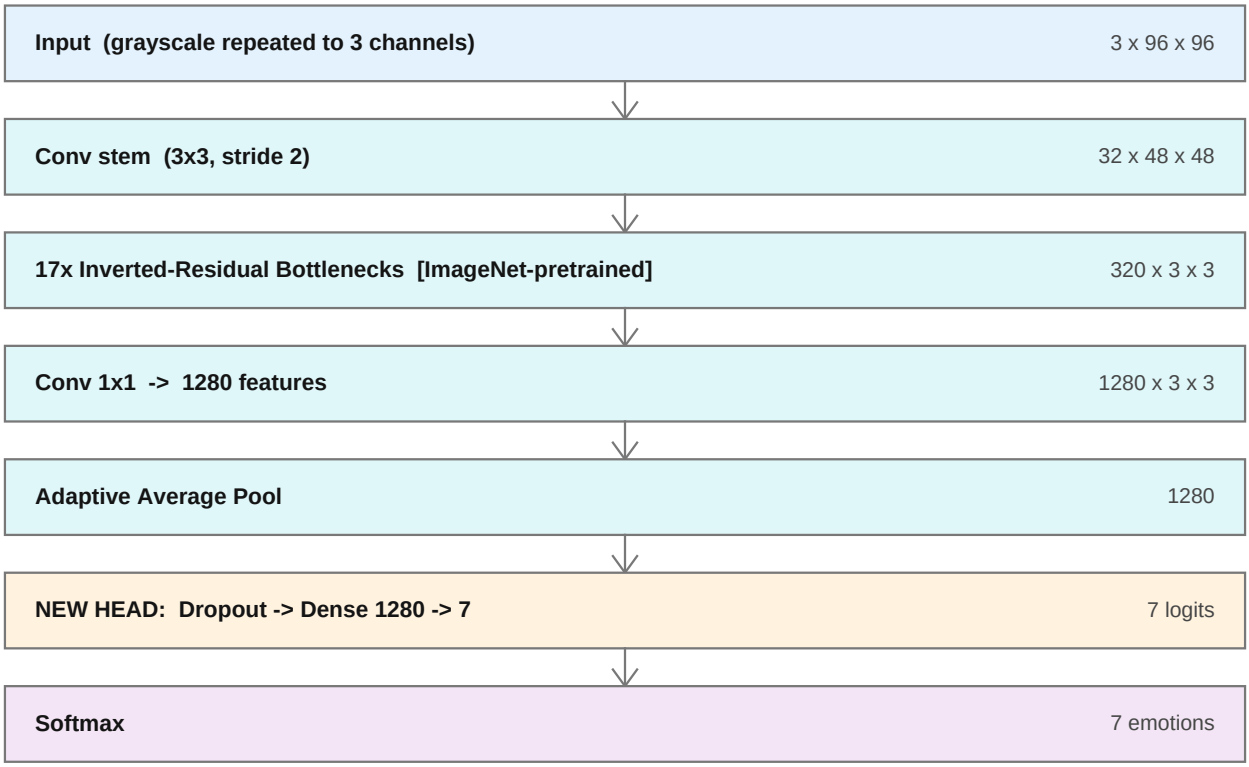



*Figure 3. Model 2 training curves - note the long flat start (SGD lr too high), then late recovery that runs out of epochs.*

### Model 3 - MobileNetV2 Transfer Learning (best_fer_model.pt) [BEST]

Instead of learning from zero, we take MobileNetV2 - a compact network already trained on millions of ImageNet photos - and fine-tune it on faces. It reuses strong visual features (edges, textures, shapes), so it learns faster and better. We upscale images to 96x96 and repeat the grayscale channel to 3 channels so the pretrained weights apply, then swap the final layer for our 7-emotion head.




Trained with AdamW + cosine decay + class-weighted cross-entropy for 20 epochs. Result: test accuracy 67.1%, macro-F1 0.661 - the best model, and the lightest (2.23M parameters vs 4.82M for the custom CNN). Saved as best_fer_model.pt for inference.

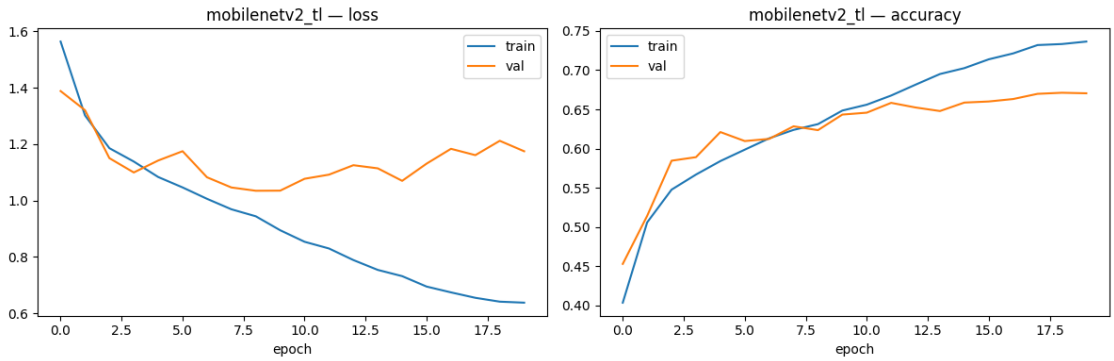



*Figure 4. Model 3 training curves - fast convergence; validation loss rises slightly late (mild overfitting) but accuracy keeps improving.*

## 5. How training works (shared setup)

- Mixed precision (AMP) for roughly 2x faster training on the T4 GPU.
- Early stopping: training halts when validation accuracy stops improving.
- Checkpointing: the epoch with the best validation accuracy is saved automatically.
- Model selection uses VALIDATION accuracy, so the test set stays an honest, unseen measure.

# 6. Results

## Side-by-side comparison (all three models)

| Model | Val acc | Test acc | Macro-F1 | Epochs | Params (M) | Time (min) |
|---|---|---|---|---|---|---|
| MobileNetV2 (TL) *best | 0.671 | 0.671 | 0.661 | 20 | 2.23 | 16.2 |
| Custom CNN baseline | 0.641 | 0.635 | 0.588 | 40 | 4.82 | 22.2 |
| Custom CNN focal+SGD | 0.533 | 0.532 | 0.474 | 40 | 4.82 | 21.3 |

Macro-F1 is the fair metric under imbalance: it averages performance across all classes equally, so it exposes models that ignore rare emotions (plain accuracy can hide that). Transfer learning wins on every metric while using the fewest parameters and the least training time.

## Per-class report (best model: MobileNetV2)

| Emotion | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| angry | 0.575 | 0.619 | 0.596 | 958 |
| disgust | 0.712 | 0.712 | 0.712 | 111 |
| fear | 0.532 | 0.446 | 0.485 | 1024 |
| happy | 0.896 | 0.843 | 0.868 | 1774 |
| neutral | 0.576 | 0.738 | 0.647 | 1233 |
| sad | 0.587 | 0.470 | 0.522 | 1247 |
| surprise | 0.754 | 0.842 | 0.796 | 831 |
| macro avg | 0.662 | 0.667 | 0.661 | 7178 |

"happy" and "surprise" are easiest; "fear" and "sad" are hardest and are most often confused with each other (and with "neutral"), as the confusion matrix shows.

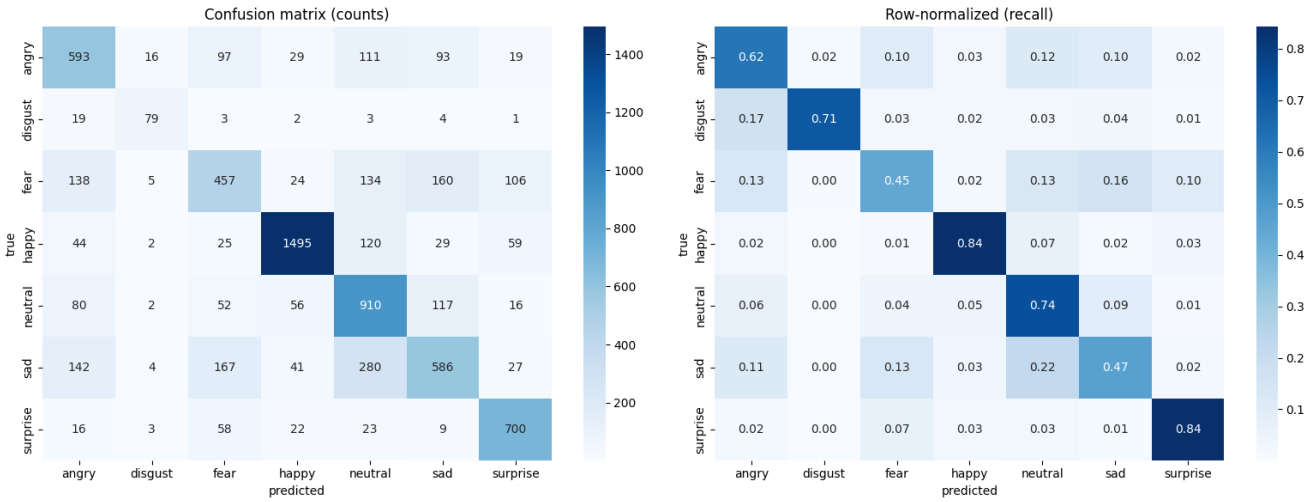



*Figure 5. Confusion matrix of the best model - counts (left) and row-normalized recall (right). The bright diagonal is correct predictions; off-diagonal cells show that fear/sad leak into neutral and into each other.*


*Figure 6. Example misclassified test faces (T = true label, P = predicted). Many are genuinely ambiguous even to a human - a direct effect of FER2013 label noise.*

## 7. How to run it

- Open FER_Colab_Notebook.ipynb in Google Colab and set Runtime -> T4 GPU.
- To retrain everything: run all cells top to bottom (about 30 min; a QUICK_RUN flag shortens it).
- To skip training: clone the repo in Colab, set SAVE_PATH to wieghts/best_fer_model.pt, and run the inference cells.
- Section 10 lets you upload a photo; Section 11 uses the webcam, detects faces, and labels each with the predicted emotion and confidence.


*Figure 7. Single-image inference (Section 10): the predicted emotion plus a confidence bar over all seven classes.*

## 8. Key takeaways and next steps

- Transfer learning (Model 3) clearly won - pretrained features beat from-scratch CNNs even on small grayscale faces, with fewer parameters and less training time.
- Focal loss + SGD (Model 2) did NOT help here and was the weakest; the technique is real but sensitive to tuning. Benchmarking caught this - assuming would have hidden it.
- The hard classes (fear, sad) and label noise are the real ceiling, not model capacity.
- Biggest future win: cleaner labels (FER+ relabels the same images) or a richer dataset (RAF-DB) would raise accuracy more than further model tuning.
- Quick boosts: ensembling the models or test-time augmentation typically add 1-2 points.